Int. Artificial y Redes Neuronales

SOLVING PROBLEMS BY SEARCHING

Dr. Mario Angel Rico Mendez

29 de octubre de 2026

FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA



Table of Contents

1 Problem-Solving Agents

- Problem-Solving Agents
- Search Algorithms

Problem-Solvin road map

1 Problem-Solving Agents

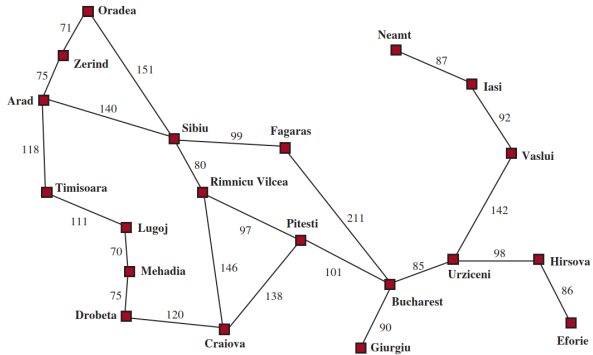


Figura 1: A simplified road map of part of Romania

Problem-Solving Agents

1 Problem-Solving Agents

- Imagine an agent on a touring vacation in Romania.
- The agent wants to:
  - Take in the sights
  - Improve its Romanian
  - Enjoy the nightlife
- The decision problem is complex.
- Suppose the agent is in Arad and has a nonrefundable ticket to fly out of Bucharest the next day.
- If the environment is unknown, the agent can only execute actions at random.

Goal Formulation

- The agent adopts the goal of reaching Bucharest.
- Goals organize behavior by limiting objectives and actions to be considered.

Problem Formulation

- The agent devises a description of states and actions to reach the goal.
- This is an abstract model of the relevant part of the world.

Search

- Before taking real-world actions, the agent simulates sequences of actions in its model.
- It searches until it finds a sequence of actions that reaches the goal.
- Such a sequence is called a **solution**.

Formal Definition of a Search Problem

A search problem consists of:

- **State Space**: A set of possible states the environment can be in.
- **Initial State**: The state the agent starts in (e.g., Arad).
- **Goal States**: One or more goal states (e.g., Bucharest).
- **Actions**: A set of actions available to the agent. Given a state s, ACTIONS(s) returns the applicable actions.
- **Transition Model**: Describes what each action does. RESULT(s, a) returns the state resulting from action a in state s.
- **Action Cost Function**: ACTION-COST(s, a, s') or $c(s, a, s')$ gives the numeric cost of applying action a in state s to reach state s'.

Paths and Solutions

- A sequence of actions forms a **path**.
- A solution is a path from the initial state to a goal state.
- Action costs are additive: the total cost of a path is the sum of individual action costs.
- An **optimal solution** has the lowest path cost among all solutions.

State Space Representation

- The state space can be represented as a graph:
  - **Vertices**: States
  - **Directed Edges**: Actions
- Example: In route-finding, cities are states, and roads are actions.

Example: Route-Finding in Romania

- Initial State: Arad
- Goal State: Bucharest
- Actions: Travel from one city to another
- Transition Model: RESULT(s, a) returns the destination city.
- Action Cost: Distance in miles between cities.
- Solution: A path from Arad to Bucharest with the lowest total distance.

Section 1.1

Formulating problems

Formulating Problems

- Problem formulation is an abstract mathematical description, not the real world.
- Example: The state description .Aradïs a simplification of a real cross-country trip.
- Irrelevant details are left out to focus on the problem (e.g., finding a route to Bucharest).
- This process is called **abstraction**.
- A good problem formulation has the right level of detail.

Abstraction

- Abstract states and actions correspond to large sets of detailed world states and action sequences.
- Abstraction is valid if:
  - Every detailed state in an abstract state (e.g., ïn Arad") has a detailed path to another abstract state (e.g., ïn Sibiu").
- This ensures that abstract solutions can be elaborated into detailed solutions.

Example Problems

- Two types of problems:
  - **Standardized Problems**: Illustrate problem-solving methods (e.g., grid world).
  - **Real-World Problems**: Idiosyncratic and specific to real-world scenarios (e.g., robot navigation).

Standardized Problems: Grid World

- A 2D rectangular array of square cells.
- Agents can move to adjacent cells (horizontally, vertically, or diagonally).
- Example: Vacuum World.

Vacuum World as a Grid Problem

- **States**:
  - Describes which objects are in which cells.
  - Objects: Agent and dirt.
  - In a 2-cell world: $2 \cdot 2 \cdot 2 = 8$ states.
  - In an n-cell world: $n \cdot 2^n$ states.
- **Initial State**: Any state can be the starting point.
- **Actions**:
  - 2-cell world: Suck, Move Left, Move Right.
  - Multi-cell world: Add Upward, Downward, or egocentric actions (Forward, Backward, TurnRight, TurnLeft).

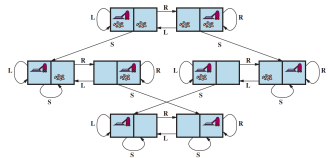


Figura 2: The state-space graph for the two-cell vacuum world

The state-space graph for the two-cell vacuum world

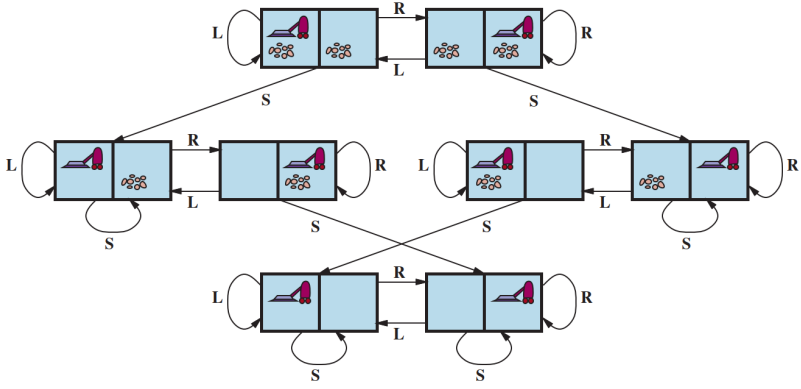


Figura 3: The state-space graph for the two-cell vacuum world

Vacuum World: Transition Model

- **Suck**: Removes dirt from the agent's cell.
- **Forward**: Moves the agent ahead one cell in the direction it is facing (unless it hits a wall).
- **Backward**: Moves the agent in the opposite direction.
- **TurnRight/TurnLeft**: Changes the agent's direction by 90°.

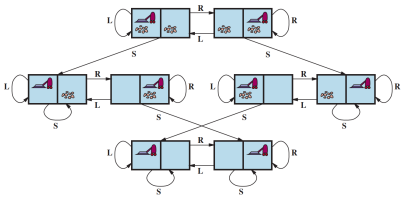


Figura 4: The state-space graph for the two-cell vacuum world

Vacuum World: Goal States and Action Cost

- **Goal States**: All cells are clean.
- **Action Cost**: Each action costs 1.

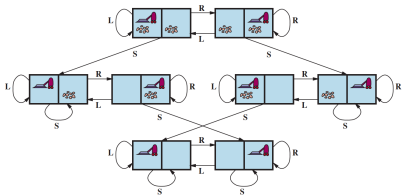


Figura 5: The state-space graph for the two-cell vacuum world

Section 1.2

Sliding-Tile Puzzle (8-Puzzle)

Sliding-Tile Puzzle (8-Puzzle)

- Tiles are arranged in a grid with one or more blank spaces.
- Tiles can slide into the blank space.
- The goal is to reach a specified goal state.

Standard Formulation of the 8-Puzzle

- **States**:
  - A state description specifies the location of each tile.
- **Initial State**:
  - Any state can be the initial state.
  - A parity property partitions the state space: only half of the initial states can reach a given goal.
- **Actions**:
  - The blank space can move Left, Right, Up, or Down.
  - If the blank is at an edge or corner, some actions are not applicable.

Standard Formulation (Continued)

- **Transition Model**:
  - Maps a state and action to a resulting state.
  - Example: Applying **Left** to the start state switches the 5 and the blank.
- **Goal State**:
  - Typically, the goal state has the numbers in order.
- **Action Cost**:
  - Each action costs 1.

Abstraction in Problem Formulation

- Every problem formulation involves abstractions.
- In the 8-Puzzle:
  - Actions are abstracted to their beginning and final states.
  - Intermediate locations (where the tile slides) are ignored.

Example: 8-Puzzle

- Start State:

2	5	3
7	1	
4	8	6

- Goal State:

1	2	3
4	5	6
7	8	

- Actions: Move the blank space Left, Right, Up, or Down.
- Transition: Moving the blank Left from the start state swaps the 5 and the blank.

Section 1.3

Real-World Problems

Robot Navigation

- A generalization of the route-finding problem.
- Unlike route-finding (e.g., roads in Romania), robots can roam freely, creating their own paths.
- For a circular robot on a flat surface, the space is two-dimensional.
- For robots with arms and legs, the search space becomes many-dimensional (one dimension for each joint angle).

Challenges in Robot Navigation

- **Continuous Search Space**:
  - — Advanced techniques are required to make the essentially continuous search space finite.
- **Sensor and Motor Errors**:
  - — Robots must deal with errors in sensor readings and motor controls.
- **Partial Observability**:
  - — Robots often have incomplete information about the environment.
- **Dynamic Environments**:
  - — Other agents (e.g., humans, other robots) can alter the environment.

Example: Circular Robot on a Flat Surface

- Search space: 2D plane.
- Actions: Move forward, backward, left, right, or rotate.
- Goal: Navigate from a starting position to a target position while avoiding obstacles.

Example: Robot with Arms and Legs

- Search space: Many-dimensional (e.g., joint angles for arms and legs).
- Actions: Control each joint angle to achieve movement.
- Goal: Perform complex tasks like walking, grasping, or climbing.

Section 1.4

Search Algorithms

Search Algorithms

- A **search algorithm** takes a search problem as input and returns a solution or failure.
- It superimposes a **search tree** over the **state space graph**.
- The search tree forms paths from the initial state, trying to reach a goal state.
- Each **node** in the search tree corresponds to a state in the state space.
- Each edge corresponds to an **action**.
- The root of the tree corresponds to the **initial state**.

State Space vs. Search Tree

- **State Space**:
  - Describes the (possibly infinite) set of states and actions in the world.
- **Search Tree**:
  - Describes paths between states, reaching towards the goal.
  - May have multiple paths to the same state.
  - Each node has a unique path back to the root.

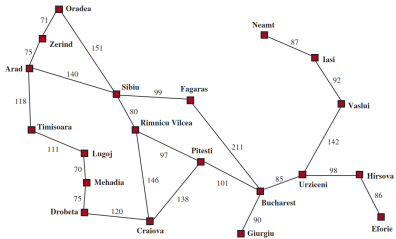


Figura 6: A simplified road map of part of Romania

Example: Path from Arad to Bucharest

- The root node of the search tree is the initial state: Arad.
- Expand the node by considering available actions and generating child nodes.
- Each child node corresponds to a resulting state (e.g., Sibiu, Timisoara, Zerind).
- Each child node has Arad as its parent node.

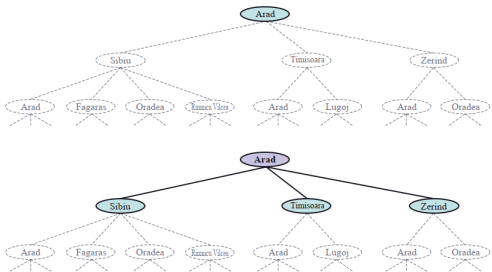


Figura 7: Two partial search trees

Choosing Nodes to Expand

- After generating **child nodes**, we must choose which node to expand next.
- This is the essence of search: following one option now and putting others aside.
- Example: Suppose we choose to expand Sibiu first.
- This generates a new set of unexpanded nodes (the frontier of the search tree).

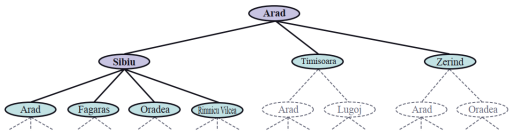


Figura 8: A partial search trees

Frontier of the Search Tree

- The frontier consists of all unexpanded nodes.
- A state is considered **reached** if a node has been generated for it.
- A state is **expanded** if its child nodes have been generated.

State Space Graphs vs. Search Trees

Each NODE in in the search tree is an entire PATH in the state space graph

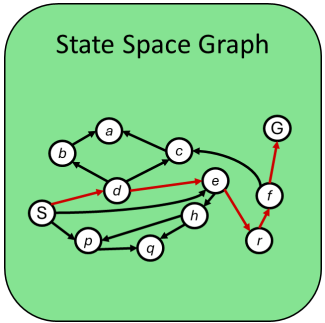


Figura 9: State Space Graph

We construct both on demand –and we construct as little as possible.

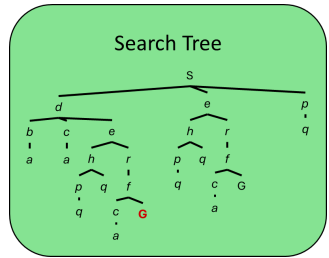


Figura 10: Search Tree

How big is its search tree (from S)?

Consider this 4-state graph:

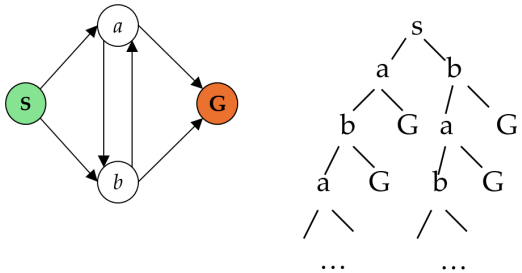


Figura 11: 4-state graph



Table of Contents

2 Search Algorithms

- Problem-Solving Agents
- Search Algorithms

Search Data Structures

- A node in the search tree is represented by:
  - **node.STATE**: The state the node corresponds to.
  - **node.PARENT**: The node that generated this node.
  - **node.ACTION**: The action applied to the parent to generate this node.
  - **node.PATH-COST**: The total cost of the path from the initial state to this node ($g(node)$).
- Following **PARENT** pointers recovers the path to the node.

Frontier Data Structure

- The frontier is stored as a queue.
- Operations on the frontier:
  - **IS-EMPTY(frontier)**: Checks if the frontier is empty.
  - **POP(frontier)**: Removes and returns the top node.
  - **TOP(frontier)**: Returns (but does not remove) the top node.
  - **ADD(node, frontier)**: Inserts a node into the queue.
- Types of queues:
  - **Priority Queue**: Used in best-first search (pops node with minimum $f(n)$).
  - **FIFO Queue**: Used in breadth-first search (pops the oldest node).
  - **LIFO Queue (Stack)**: Used in depth-first search (pops the newest node).

Reached States

- Reached states are stored in a lookup table (e.g., a hash table).
- Each key is a state, and the value is the corresponding node.
- Helps detect redundant paths and keep the best path to each state.

Redundant Paths

- A redundant path is a worse way to reach the same state.
- Example: Paths to Sibiu:
  - Arad–Sibiu (140 miles).
  - Arad–Zerind–Oradea–Sibiu (297 miles).
- Cycles (loopy paths) are a special case of redundant paths.
- Strategies to handle redundant paths:
  - Remember all reached states and keep the best path.
  - Ignore redundant paths (if rare or impossible).
  - Check for cycles but not general redundant paths.

Measuring Problem-Solving Performance

- Criteria to evaluate search algorithms:
  - **Completeness**: Guaranteed to find a solution if one exists.
  - **Cost Optimality**: Finds the solution with the lowest path cost.
  - **Time Complexity**: Time taken to find a solution (measured in states/actions considered).
  - **Space Complexity**: Memory required to perform the search.

FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Q&A

Thank you for your attention

Russell, S. J., & Norvig, P. (2016). Artificial intelligence: a modern approach. Pearson.
Hunt, E. B. (2014). Artificial intelligence. Academic Press.
Ertel, W. (2024). Introduction to artificial intelligence. Springer Nature.